



7. ABSTRACT CLASS AND INTERFACE

Obet Mubarak



Pilihan :

1. Abstract Class

2. Interface

ABSTRACT CLASS

- Abstract class adalah kelas yang di dalamnya terdapat method abstrak.
- Method abstrak adalah method yang hanya terdiri atas definisi dan tidak memiliki implementasi.

- Contoh :

```
public void printInfo();
```

- Tujuan membuat kelas abstrak adalah menyediakan kelas yang memiliki method-method yang dapat digunakan bersama oleh kelas-kelas turunannya.
- Kita tidak dapat membuat objek dari kelas abstrak, tetapi kita dapat membuat variabel bertipe kelas abstrak.
- Kelas turunan dari kelas abstrak harus *meng-override* method abstrak dan melengkapinya. Jika tidak maka kelas turunan tsb juga menjadi kelas abstrak.

abstract Lingkaran3D

- double jari;
- double phi=3.14;
- + double getJari()
- + void setJari(double jari)
- + double getLuasLingkaran()
- + abstract double getLuasBangun()
- + abstract double getVolBangun()

Tabung

- double tinggi;
- cosntructor(double tinggi)
- + getTinggi()
- + setTinggi()
- + double getLuasBangun()
- + double getVolBangun()

Bola

- + double getLuasBangun()
- + double getVolBangun()



```
abstract class BangunRuangLingkaran
{
    private double jari;
    final double phi=3.14;

    public double getJari() {
        return jari;
    }

    public void setJari(double jari) {
        this.jari = jari;
    }

    double getLuasLingkaran()
    {
        return (phi*jari*jari);
    }
    abstract double getLuasBangun();
    abstract double getVolBangun();
}
```

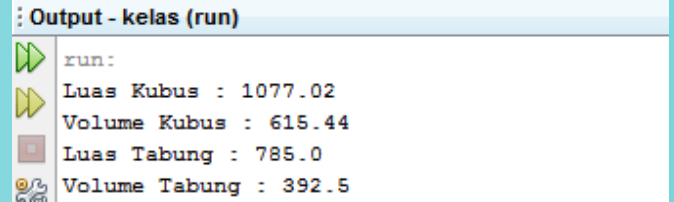
```
class Tabung extends BangunRuangLingkar
an
{
    private double tinggi;

    public Tabung(double tinggi) {
        this.tinggi = tinggi;
    }

    public double getLuasBangun()
    {
        return 2*getLuasLingkaran()*tinggi;
    }
    public double getVolBangun()
    {
        return getLuasLingkaran()*tinggi;
    }
}
```

```
class Bola extends BangunRuangLingkaran
{
    public double getLuasBangun()
    {
        return 4/3*getLuasLingkaran()*getJari();
    }
    public double getVolBangun()
    {
        return 4*getLuasLingkaran();
    }
}
```

```
public class DemoAbstractClass {  
    public static void main(String[] args) {  
        Lingkaran3D brl;  
        Bola b = new Bola();  
        Tabung t = new Tabung(5);  
  
        brl=b;  
        brl.setJari(7);  
        System.out.println("Luas Kubus : "+ brl.getLuasBangun());  
        System.out.println("Volume Kubus : "+brl.getVolBangun());  
  
        brl=t;  
        brl.setJari(5);  
        System.out.println("Luas Tabung : "+brl.getLuasBangun());  
        System.out.println("Volume Tabung : "+brl.getVolBangun());  
    }  
}
```



Output - kelas (run)

run:
Luas Kubus : 1077.02
Volume Kubus : 615.44
Luas Tabung : 785.0
Volume Tabung : 392.5

INTERFACE

- Interface adalah struktur mirip kelas yang hanya terdiri dari : konstanta dan abstract method.
- Merupakan jenis khusus dari blok yang sepenuhnya tidak diimplementasikan jadi perlu class untuk **mengimplementasi**
- Format interface :

```
interface [nama_interface] {  
    //deklarasikan konstanta  
  
    //deklarasikan method abstrak  
}
```

INTERFACE

Variable secara otomatis akan bersifat public, final dan static

Method secara otomatis akan bersifat abstract

Kelas yang implements interface **wajib** mengoverride seluruh method yang ada di interface.

interface tidak dapat diinstansiasi tp kita dapat membuat variabel yang bertipe interface.

interface juga dapat berperan dalam POLIMORFISME.

Interface Kendaraan

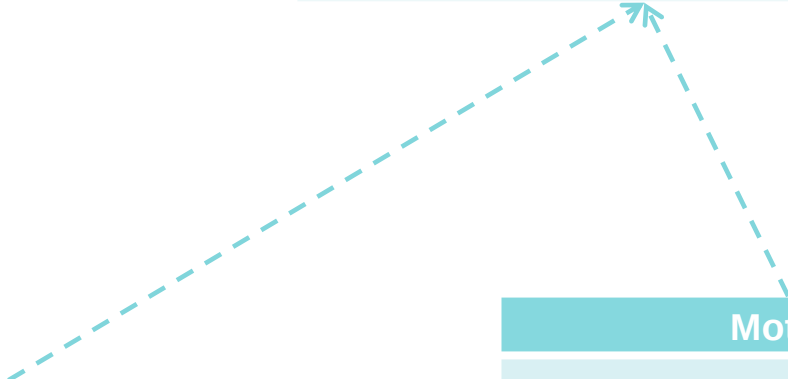
+ void start()
+ void stop()

Mobil

+ void start()
+ void stop()

Motor

+ void start()
+ void stop()



```
interface Kendaraan
{
    public void start();
    public void stop();
}
```

```
class mobil implements Kendaraan
{
    public void start()
    {
        System.out.println("menjalankan mobil");
    }
    public void stop()
    {
        System.out.println("menghentikan mobil");
    }
}
```

```
class motor implements Kendaraan
{
    public void start()
    {
        System.out.println("menjalankan motor");
    }
    public void stop()
    {
        System.out.println("menghentikan motor");
    }
}
```

```
public class Demo Mobil {  
    public static void main(String[] args) {  
        Kendaraan k ;  
        mobil m = new mobil();  
        motor t = new motor();  
  
        k=m;  
        k.start();  
        k.stop();  
  
        k=t;  
        k.start();  
        k.stop();  
    }  
}
```

Output - kelas (run)



run:



menjalankan mobil

menghentikan mobil

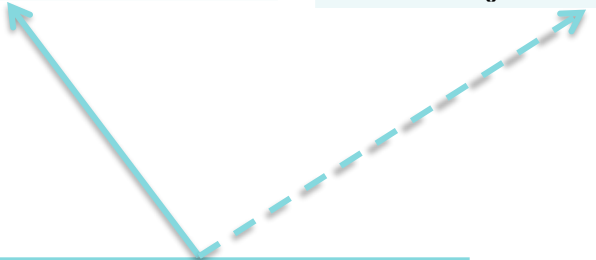
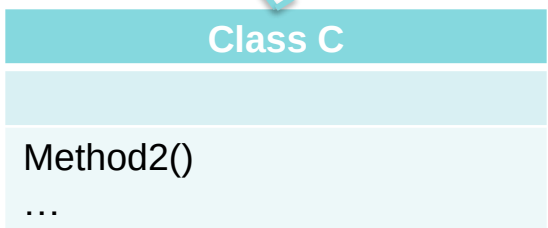
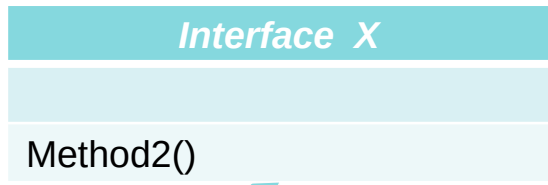
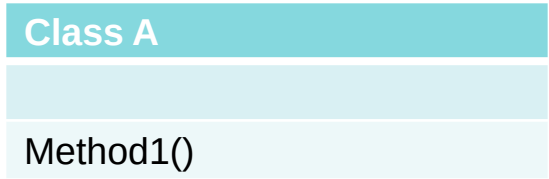


menjalankan motor



menghentikan motor

jenis	constructor	method	variabel
Interface	Tidak ada	Abstract → otomatis	(public, final dan static) → otomatis
Abstract class	Tidak ada	abstract non abstract	bebas
Class	ada	non abstract	bebas



```
Public class C extends A implements X
{
    ...
}
```

